Transformer Model

산업경영공학과 12210588 고우현

인하대학교
INHA UNIVERSITY

Operations Research and
Artificial Intelligence Lab

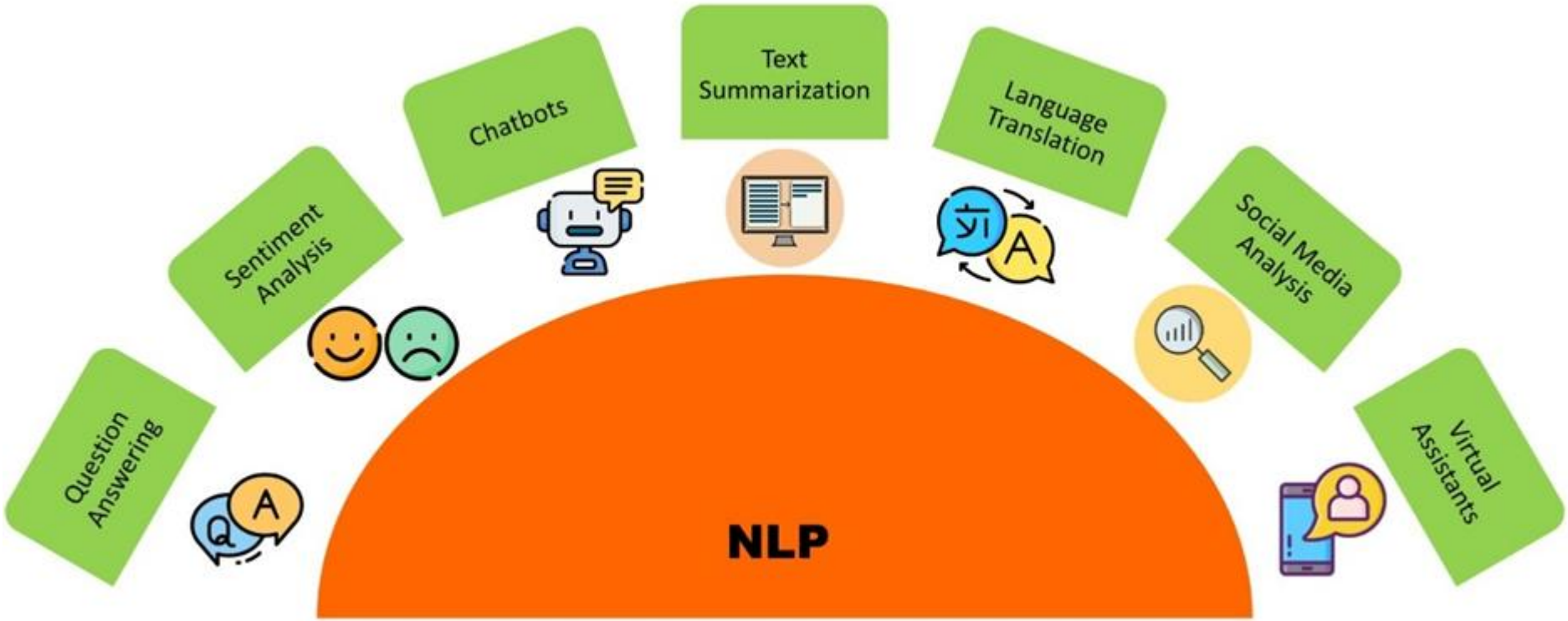
Contents

Operations Research and Artificial Intelligence Lab

- Natural Language Processing (NLP)
- Transformer Model
- Self-Attention
- Transformer Method
- Example
- Vision Transformer (ViT)

Natural Language Processing (NLP)

- 자연어 처리는 사람이 사용하는 언어를 컴퓨터가 이해하고 처리하는 기술

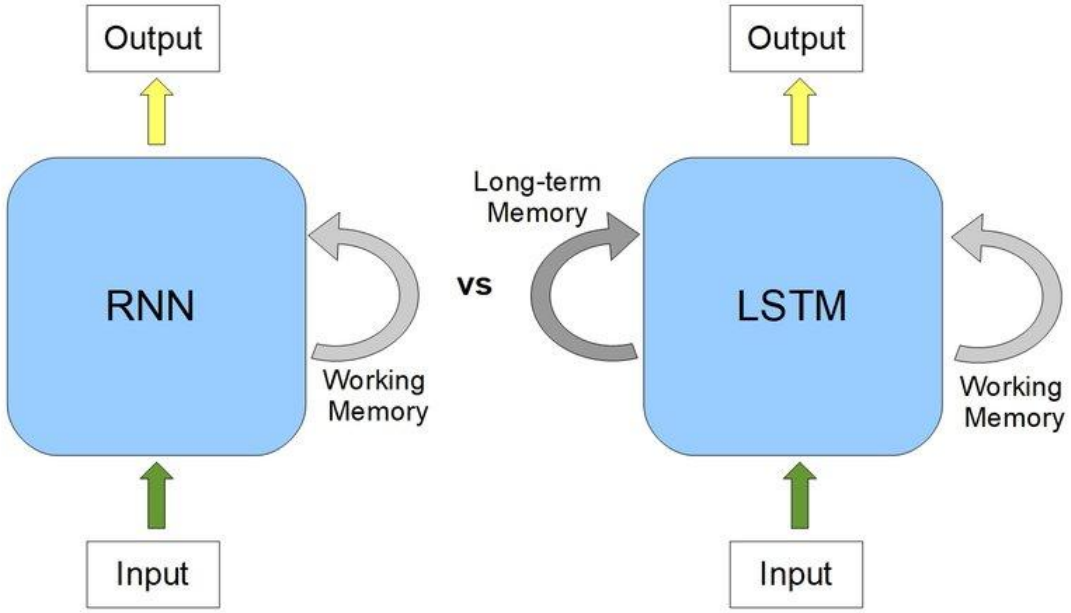


ex) : Translator, Chatbot(ChatGPT, Gemini), Search, Summary,,,

Previous NLP Model

Operations Research and Artificial Intelligence Lab

RNN과 LSTM은 문장을 순서대로, 순환 구조로 하나씩 처리



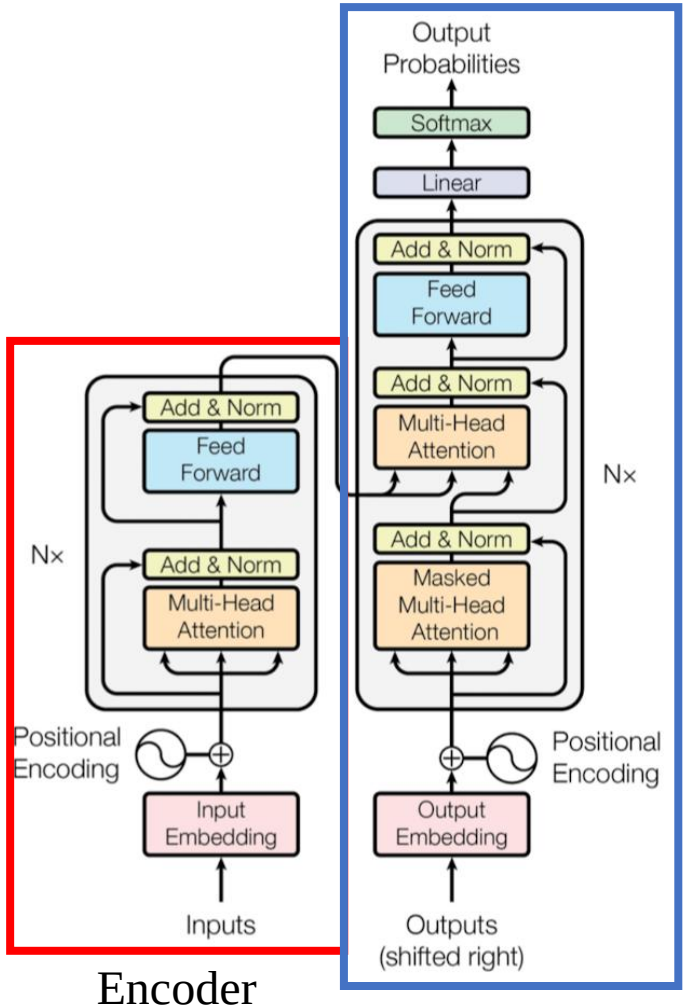
Problem

1. Parallel processing X
2. Difficulty in training
3. Slow processing speed

Transformer Model

Operations Research and Artificial Intelligence Lab

Attention Is All You Need (Vaswani et al. 2017 NeurIPS)



Encoder

Decoder

기존 RNN과 LSTM은 순차 처리한다는 단점을 보완
→ 문장을 순서대로 읽지 않고, 한 번에 병렬로 처리하여 의미를 이해

Encoder : 문장의 의미를 이해

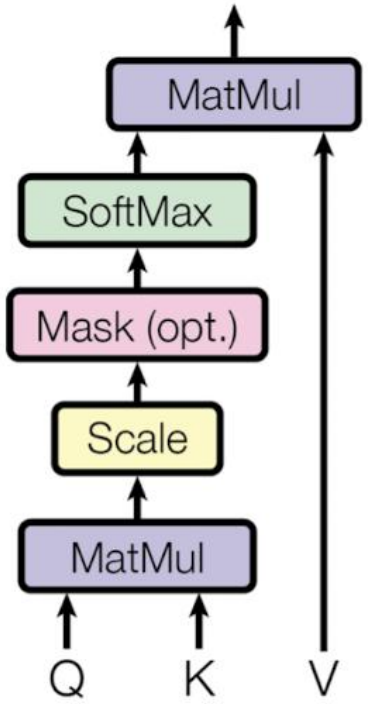
Decoder : 다음 단어를 생성

Self-Attention

Operations Research and Artificial Intelligence Lab

Self-Attention은 문장 속 중요한 단어를 스스로 찾는 과정으로 문장 속 각 단어가 다른 단어들을 얼만큼 참고해야 하는가를 계산하는 방법

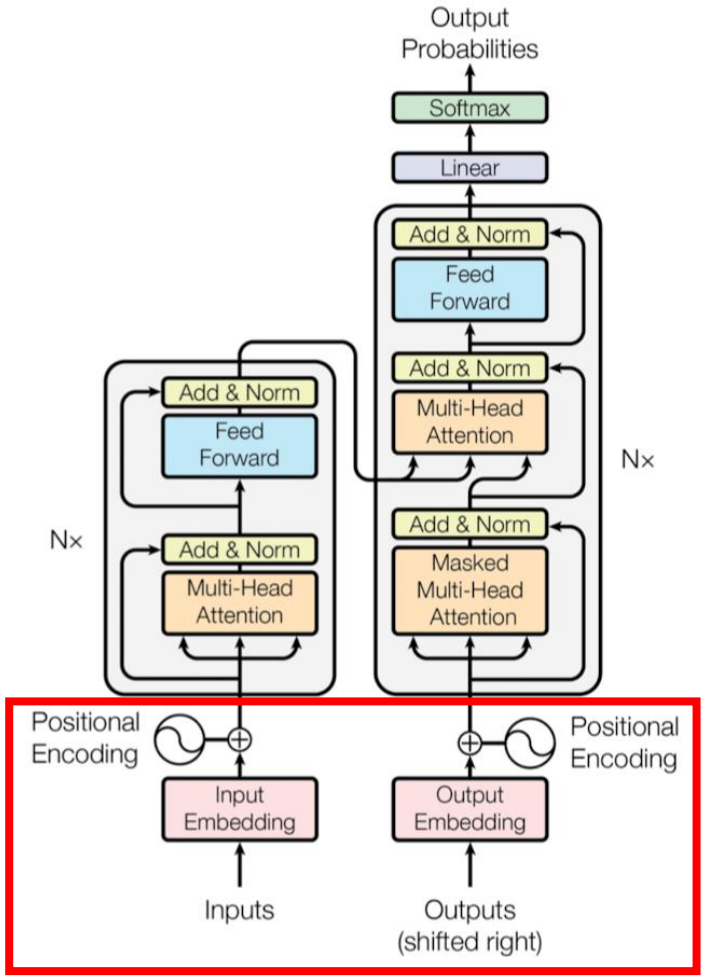
Scaled Dot-Product Attention



- Q(query)를 K(key)와 비교한 가중치를 V(value)에 반영
- Q는 찾고 싶은 정보로 질문을 던짐
- K는 비교 대상으로 다른 단어들과 비교
- V는 최종 반영될 정보로 중요한 정보만 수집

→ 결국 단어 간의 중요도를 계산해서 가중 평균을 내는 과정

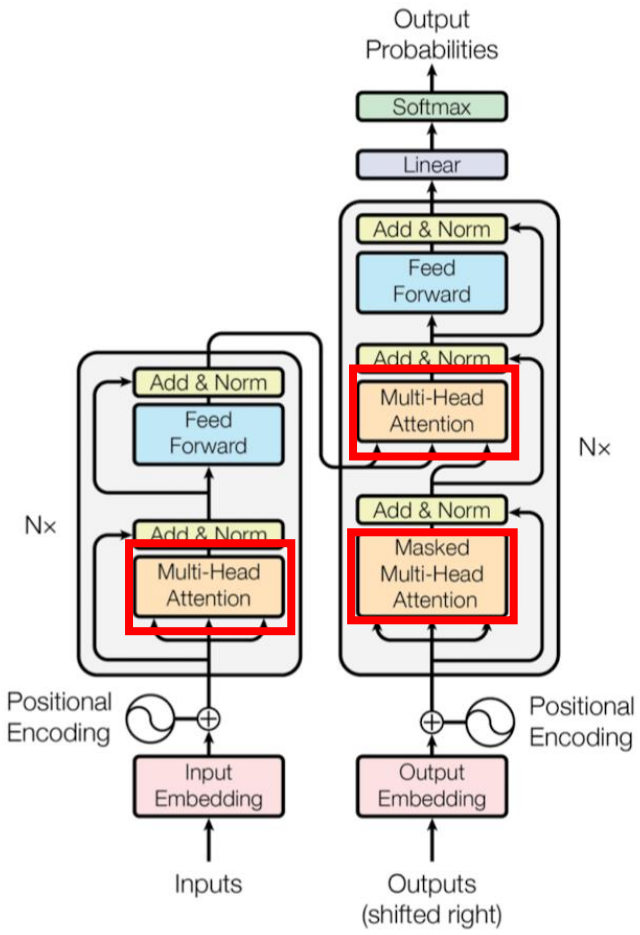
Input Embedding & Positional Encoding



단어를 이해하기 위해 벡터로 변환하는 과정 필요

1. 문장의 각 단어들을 정해진 규칙에 의해 토큰화
2. 토큰화로 가공된 숫자들을 벡터로 변환 (Input Embedding)
3. 변환된 벡터에 각 단어들의 위치 정보 추가 (Positional Encoding)

Multi-Head Attention

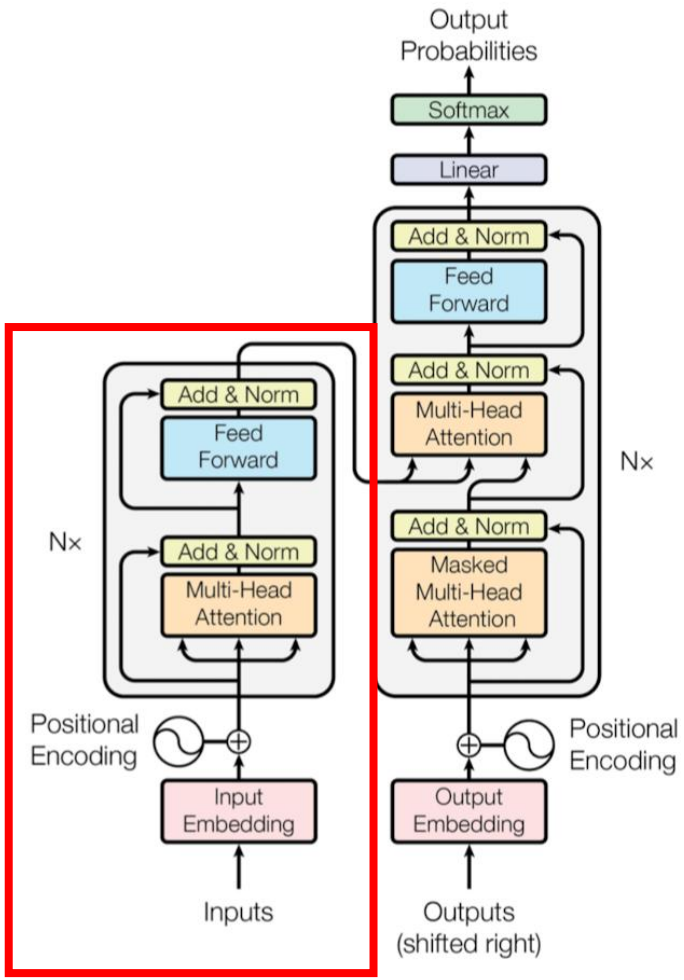


Transformer 모델에서는 Self-Attention을 한번만 진행하는 것이 아닌 여러 개를 동시에 진행

각각의 Head는 문장의 다른 관계를 주목

한 번의 Self-Attention으로는 문장의 모든 의미를 이해하기 어렵기 때문에 여러 번 진행

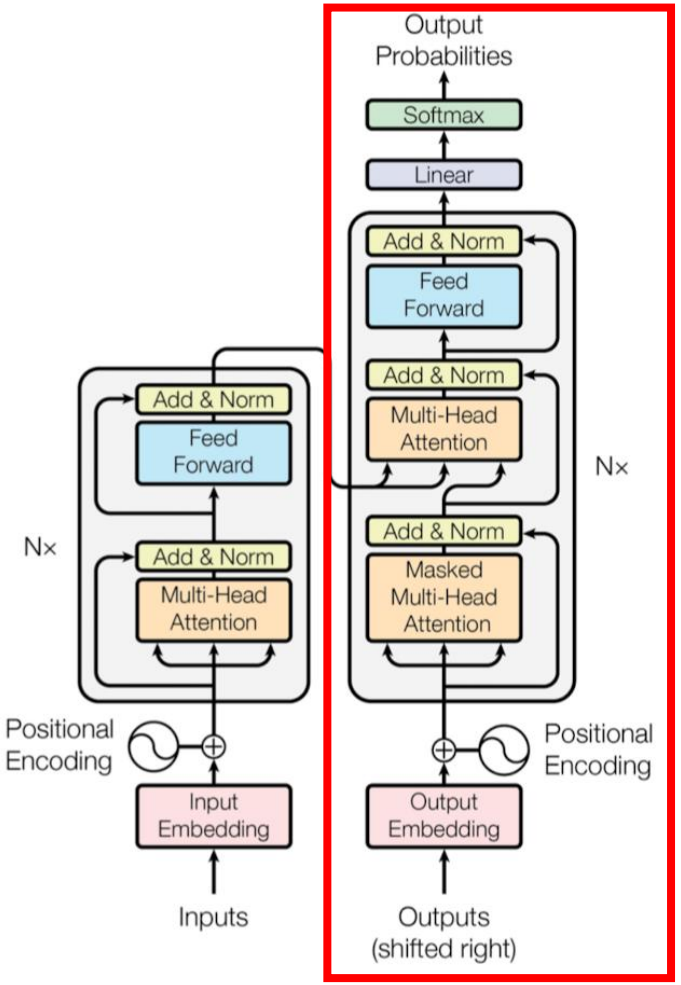
How Encoder works?



Encoder는 입력 문장을 이해한 후 의미 있는 표현으로 변환

1. Multi-Head Attention
2. Add & Norm
   - 이전 정보를 유지하고 새로 계산한 정보를 적당히 섞어 안정적으로 전달
3. Feed Forward Network(FFN)
   - 단어 벡터 하나하나를 한 번 더 정리
4. Add & Norm

How Decoder works?

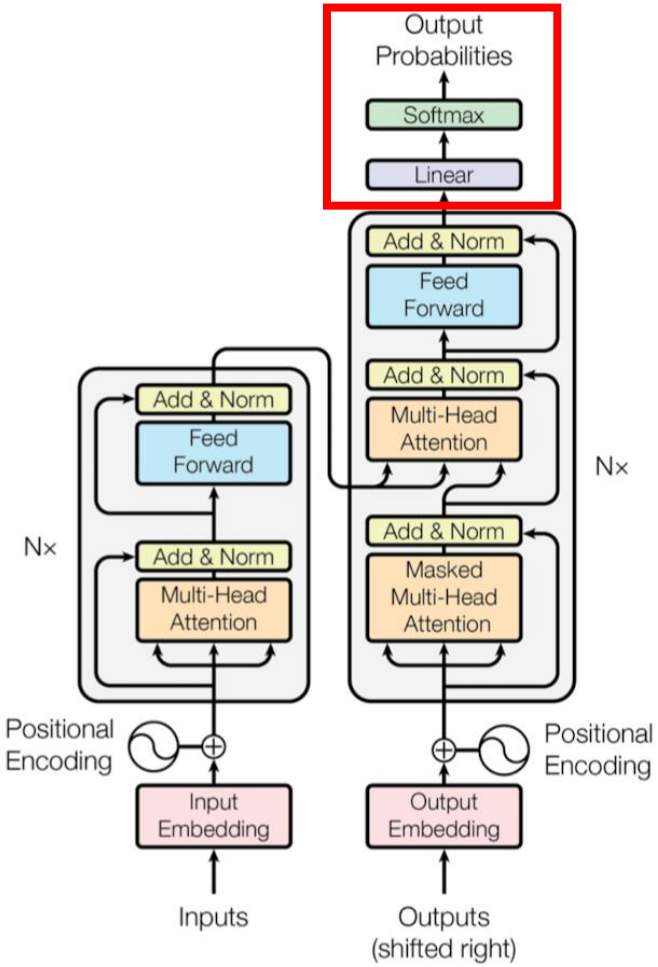


Decoder 는 Encoder가 이해한 내용을 바탕으로 한 단어씩 생성

1. Masked Multi-Head Attention
   - 아직 나오지 않은 미래 단어를 보이지 않도록 가린 구조
2. Add & Norm
3. Multi-Head Cross Attention(Encoder-Decoder Attention)
   - Decoder가 Encoder의 출력을 참고하여 Q는 Decoder, K와 V는 Encoder 를 적용해 Decoder가 질문을 제공하고 Encoder는 정보를 제공하는 구조
4. Add & Norm
5. Feed Forward Network
6. Add & Norm

Decoder는 다음에 나올 단어의 확률을 출력

Transformer model output



1. Decoder 출력
2. Linear
   - Decoder의 출력을 바탕으로 각 단어가 될 가능성 점수 계산
3. Softmax
   - 앞서 계산한 점수를 확률화
4. 다음 단어
   - 최대값을 갖는 확률의 단어를 선택해 출력

Transformer code example

Operations Research and Artificial Intelligence Lab

Hugging Face : 사전 학습된 Transformer 모델을 쉽게 불러와서 바로 사용할 수 있는 AI 모델 플랫폼

적용 모델 : ‘facebook/nllb-200-distilled-600M’

Meta(Facebook)가 만든 Transformer 기반 다국어 번역 모델

```
from transformers import pipeline

translator = pipeline("translation", model="facebook/nllb-200-distilled-600M")

en_text = "The Transformer model is the foundation of modern AI."
result = translator(en_text, src_lang="eng_Latn", tgt_lang="kor_Hang")

print(result[0]['translation_text'])
```

```
트랜스포머 모델은 현대 인공지능의 기초입니다.
```

Transformer model example

Transformer를 대표적으로 활용한 모델 : BERT, GPT, T5, ViT

1. BERT : 문장을 이해하는 모델(Encoder)

   → 검색 엔진 문장 이해, 문서 분류, 감정 분석

2. GPT : 문장을 생성하는 모델(Decoder)

   → 챗봇, 글 자동 작성, 대화형 AI 비서

3. T5 : 번역, 요약을 하나의 방식으로 처리

   → 기계 번역, 문서 요약, 문장 변환

4. Vision Transformer (ViT) : Transformer를 이미지에 적용한 모델

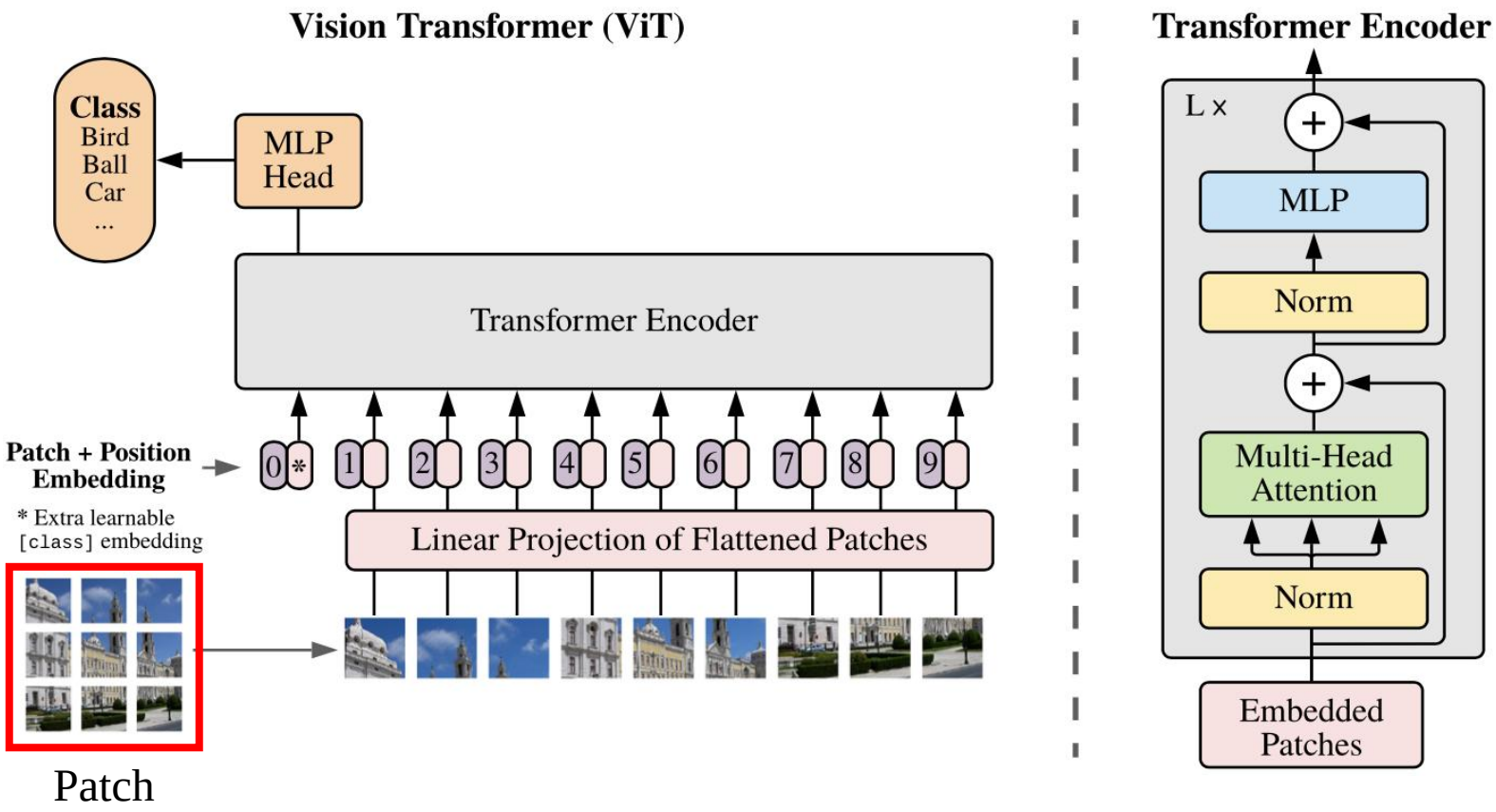
   → 이미지 분류, 얼굴 인식, 자율주행 영상 인식

Vision Transformer (ViT)

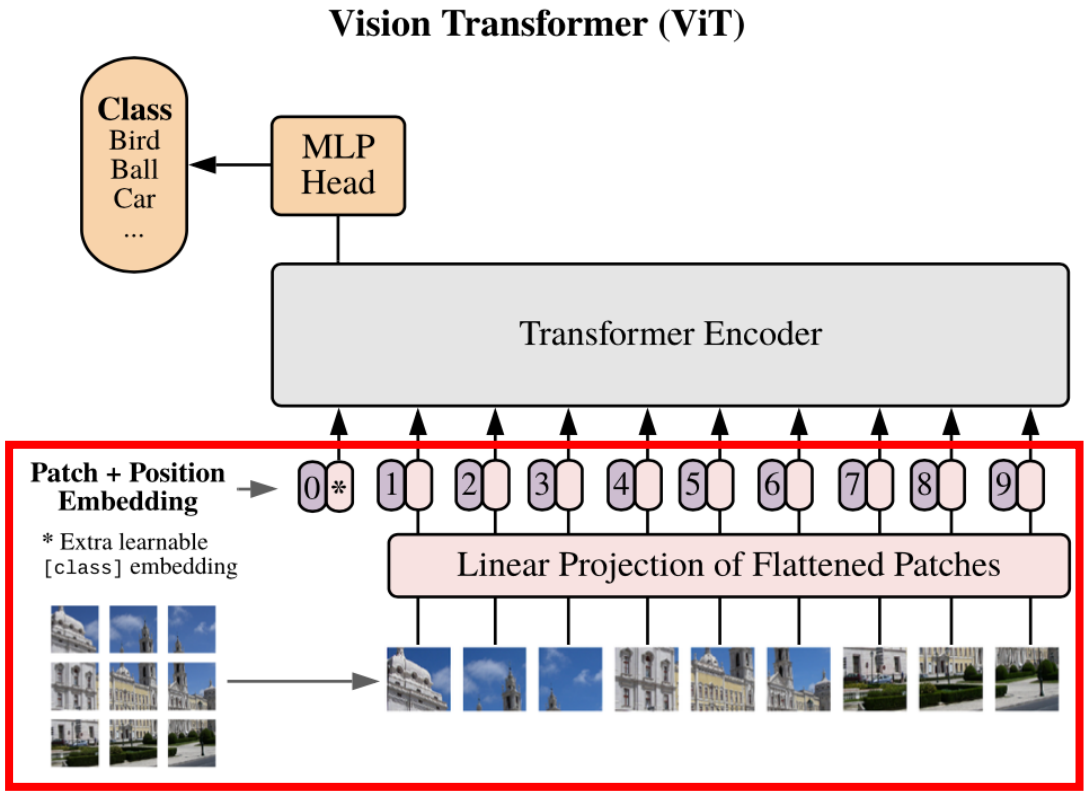
Operations Research and Artificial Intelligence Lab

Vision Transformer : Transformer 모델을 이미지에 적용한 모델

Transformer는 문장을 토큰으로, ViT는 이미지를 패치로 나눠 학습



Patch Embedding & Positional Encoding



- Patch Embedding

  이미지를 일정 크기의 패치로 나눠 토큰화 한 후 벡터로 변환

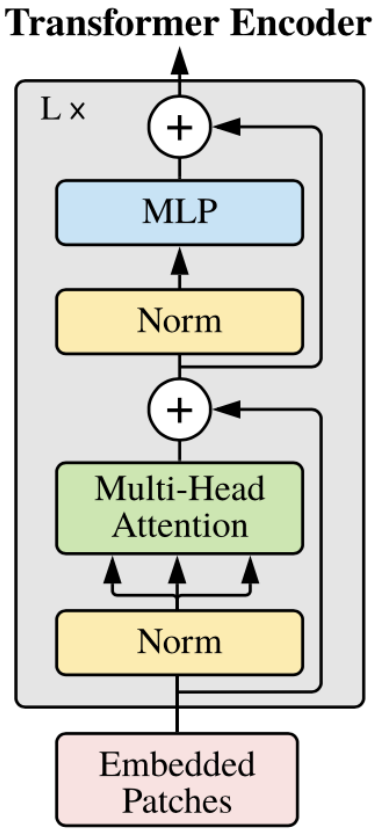
- [CLS] 토큰 추가

  이미지 전체를 대표하는 요약 토큰

- Positional Encoding

  각 패치가 이미지의 어느 위치에 있는지 표현

How ncoder work in ViT?

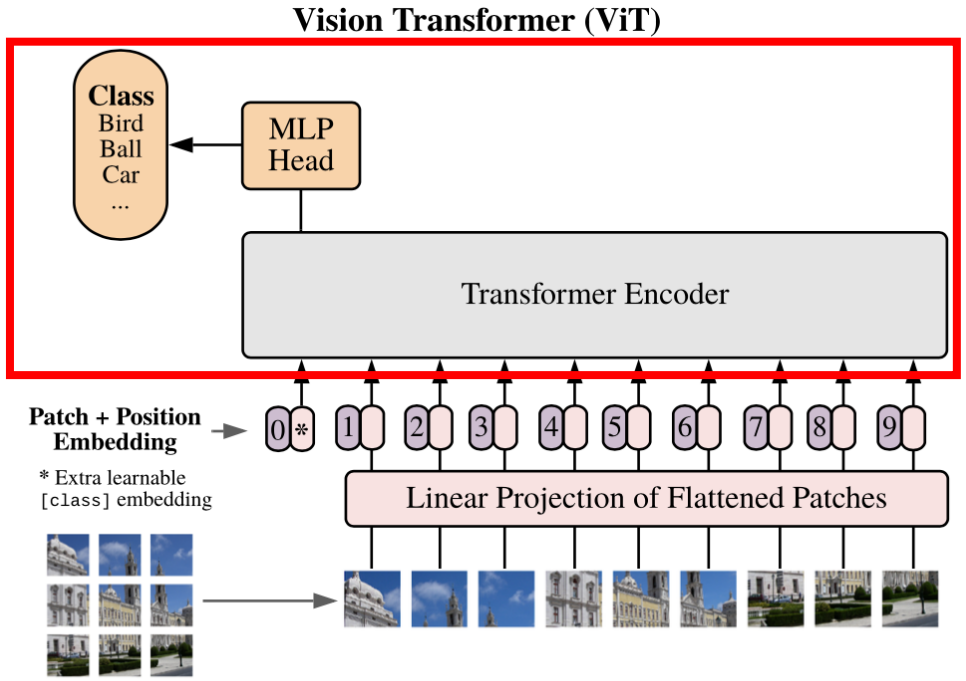


- 이미지 패치들을 전체 이미지 정보를 반영한 표현으로 변환
- [CLS] 토큰에 이미지 전체 정보를 요약

과정

1. Norm
2. Multi-Head Attention
3. Add
4. Norm
5. MLP : 단어 벡터 하나하나를 한 번 더 정리
6. Add

ViT model result



- Transformer Encoder : [CLS] 토큰 추출
- MLP Head : 클래스별 점수 계산
- Classifier : 클래스별 점수를 기반으로 분류

ViT code example

Operations Research and Artificial Intelligence Lab

적용 모델 : ‘google/vit-base-patch16-224’

Google에서 만든 Vision Transformer 기반 이미지 분류 모델

```python
from transformers import pipeline

classifier = pipeline(task="image-classification", model="google/vit-base-patch16-224")

url = "https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/pipeline-cat-chonk.jpeg"

results = classifier(url)

for result in results:
    print(f"이것은 {result['label']}일 확률이 {round(result['score']*100, 2)}%입니다.")
```

```
이것은 lynx, catamount일 확률이 43.35%입니다.
이것은 cougar, puma, catamount, mountain lion, painter, panther, Felis concolor일 확률이 3.48%입니다.
이것은 snow leopard, ounce, Panthera uncia일 확률이 3.24%입니다.
이것은 Egyptian cat일 확률이 2.39%입니다.
이것은 tiger cat일 확률이 2.29%입니다.
```

CNN vs ViT

- 기존 이미지 인식에서는 이미지의 국소적인 특징을 잘 포착하는 CNN이 대세
- 그러나 CNN은 가까운 영역을 먼저 보고 멀리 있는 정보를 이해하는데 한계 존재(지역적)
- 이에 대한 대책으로 이미지 전체를 동시에 바라봐 멀리 떨어진 영역도 볼 수 있고 대용량 데이터에 강한 ViT를 사용(전역적)

Comparison Detail

Operations Research and Artificial Intelligence Lab

Feature	Vision Transformer	CNN
특징 추출	이미지 패치 간의 Self-attention 사용	합성곱 필터를 사용하여 국소적 특징 추출
전역적 맥락	단일 레이어에서 패치 간의 전역적 관계 포착	국소적 특징으로부터 전역적 이해를 쌓기 위해 깊은 레이어 필요
이미지 처리	이미지를 겹치지 않는 패치 단위로 분할	필터를 사용하여 이미지 스캔
위치 인코딩	패치 위치 인식을 위해 명시적인 위치 인코딩 필요	합성곱 구조를 통해 위치 정보를 인코딩
모델 복잡도	Attention으로 인해 연산 비용이 높음	일반적으로 ViT보다 연산 비용이 낮음
데이터 요구량	성능을 내기 위해 대규모 데이터셋 필요	소규모 및 대규모 데이터셋 모두에서 우수한 성능
확장성	데이터셋과 모델이 커질수록 확장성이 매우 뛰어남	확장성은 좋으나 소규모 데이터셋에서 과적합 위험
소규모 데이터셋 성능	소규모 데이터셋에서 과적합되는 경향이 있음	소규모 데이터셋에서 더 나은 성능 발휘
해석 가능성	Attention으로 인해 해석이 어려움	학습된 필터를 통해 해석이 비교적 쉬움

Q&A